

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

SYNCHRONOUS VS. ASYNCHRONOUS GPU EXECUTION

Lab 5

Matt Delalieux (r0890338)
Radi Miah (r1012080)

Academic year 2025–2026

Contents

1 Implementation **2**
1.1 Reduction Kernels 2
1.2 Synchronous Approach 3
1.3 Asynchronous Approach 3

2 Results **4**

3 Analysis **6**
3.1 Why Asynchronous Execution is Faster 6
3.2 Observed Speedup and Limiting Factors 6

1 Implementation

1.1 Reduction Kernels

The same reduction logic from Lab 2 was reused: each kernel loads a portion of the input into shared memory, performs a semi-parallel reduction using a stride loop, and writes one partial result per block back to shared memory. Then the CPU will finish the final reduction over the blocks.

```
__global__ void reduceSum(float *input, float *block_results,
                          int n, int itemsPerBlock) {
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
    int blockStart = blockIdx.x * itemsPerBlock;
    float localVal = 0.0f;
    for (int i = tid; i < itemsPerBlock && (blockStart + i) < n; i += blockDim.x)
        localVal += input[blockStart + i];
    sdata[tid] = localVal;
    __syncthreads();
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if (tid % (2 * stride) == 0 && (tid + stride) < blockDim.x)
            sdata[tid] += sdata[tid + stride];
        __syncthreads();
    }
    if (tid == 0) block_results[blockIdx.x] = sdata[0];
}

__global__ void reduceProd(float *input, float *block_results,
                           int n, int itemsPerBlock) {
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
    int blockStart = blockIdx.x * itemsPerBlock;
    float localVal = 1.0f;
    for (int i = tid; i < itemsPerBlock && (blockStart + i) < n; i += blockDim.x)
        localVal *= input[blockStart + i];
    sdata[tid] = localVal;
    __syncthreads();
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if (tid % (2 * stride) == 0 && (tid + stride) < blockDim.x)
            sdata[tid] *= sdata[tid + stride];
        __syncthreads();
    }
    if (tid == 0) block_results[blockIdx.x] = sdata[0];
}

__global__ void reduceMin(float *input, float *block_results,
                          int n, int itemsPerBlock) {
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
    int blockStart = blockIdx.x * itemsPerBlock;
    float localVal = FLT_MAX;
    for (int i = tid; i < itemsPerBlock && (blockStart + i) < n; i += blockDim.x) {
        float val = input[blockStart + i];
        if (val < localVal) localVal = val;
    }
    sdata[tid] = localVal;
    __syncthreads();
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if (tid % (2 * stride) == 0 && (tid + stride) < blockDim.x)
            if (sdata[tid + stride] < sdata[tid]) sdata[tid] = sdata[tid + stride];
        __syncthreads();
    }
    if (tid == 0) block_results[blockIdx.x] = sdata[0];
}

__global__ void reduceMax(float *input, float *block_results,
                          int n, int itemsPerBlock) {
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
    int blockStart = blockIdx.x * itemsPerBlock;
    float localVal = -FLT_MAX;
    for (int i = tid; i < itemsPerBlock && (blockStart + i) < n; i += blockDim.x) {
        float val = input[blockStart + i];
```

```
        if (val > localVal) localVal = val;
    }
    sdata[tid] = localVal;
    __syncthreads();
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if (tid % (2 * stride) == 0 && (tid + stride) < blockDim.x)
            if (sdata[tid + stride] > sdata[tid]) sdata[tid] = sdata[tid + stride];
        __syncthreads();
    }
    if (tid == 0) block_results[blockIdx.x] = sdata[0];
}
```

Listing 1: All four reduction kernels

1.2 Synchronous Approach

In the synchronous implementation, each operation follows the same four steps in strict sequence: (1) generate the array on the CPU, (2) transfer it to the GPU via a blocking `cudaMemcpy`, (3) launch the reduction kernel and wait with `cudaDeviceSynchronize`, and (4) copy the results back with another blocking `cudaMemcpy`. The CPU is idle during steps 2-4, and the next operation only begins once all four steps of the previous one have completed.

```
void runSync(BenchContext& ctx, int n, int blocks, float results[4]) {
    int itemsPerBlock = (n + blocks - 1) / blocks;
    int shared = THREADS * sizeof(float);

    // --- SUM ---
    generateArray(ctx.h_arr_sync, n, 1);
    cudaMemcpy(ctx.d_input_sync, ctx.h_arr_sync,
               n * sizeof(float), cudaMemcpyHostToDevice);   // blocking
    reduceSum<<<blocks, THREADS, shared>>>(
    ctx.d_input_sync, ctx.d_block_sync, n, itemsPerBlock);
    cudaDeviceSynchronize();
    cudaMemcpy(ctx.h_block_sync, ctx.d_block_sync, blocks * sizeof(float),
        cudaMemcpyDeviceToHost);
    results[0] = finishSum(ctx.h_block_sync, blocks);

    // --- PROD / MIN / MAX follow the same pattern ---
    ...
}
```

Listing 2: Synchronous workflow (excerpt for the sum operation)

1.3 Asynchronous Approach

The asynchronous implementation offloads work to the GPU without blocking the CPU. Each operation gets its own stream, an ordered call stack of GPU operations that executes independently of other streams.

Array 0 is generated first, then `cudaMemcpyAsync` copies it to the GPU, the kernel is launched, and `cudaMemcpyAsync` copies the result back. This all happened on stream 0 without waiting. The CPU immediately continues generating array 1, which is then enqueued on stream 1 in the same way. This repeats for arrays 2 and 3. Because the CPU does not wait for the GPU, the CPU and GPU run in parallel with less blocking. After all four streams are enqueued, `cudaStreamSynchronize` is called on each stream before reading the results.

```
void runAsync(BenchContext& ctx, int n, int blocks, float results[4]) {
    int itemsPerBlock = (n + blocks - 1) / blocks;
    int shared = THREADS * sizeof(float);

    // Generate array 0 first (nothing to overlap yet)
    generateArray(ctx.h_arr_async[0], n, 1);

    // Enqueue sum pipeline on stream 0 asynchronously
    cudaMemcpyAsync(ctx.d_input_async[0], ctx.h_arr_async[0],
                    n * sizeof(float), cudaMemcpyHostToDevice, ctx.streams[0]);
    reduceSum<<<blocks, THREADS, shared, ctx.streams[0]>>>(
        ctx.d_input_async[0], ctx.d_block_async[0], n, itemsPerBlock);
    cudaMemcpyAsync(ctx.h_block_async[0], ctx.d_block_async[0],
```

```
                    blocks * sizeof(float), cudaMemcpyDeviceToHost, ctx.streams[0]);

    // While stream 0 runs on GPU, generate array 1 on CPU
    generateArray(ctx.h_arr_async[1], n, 2);
    cudaMemcpyAsync(ctx.d_input_async[1], ctx.h_arr_async[1],
                    n * sizeof(float), cudaMemcpyHostToDevice, ctx.streams[1]);
    reduceProd<<<blocks, THREADS, shared, ctx.streams[1]>>>(
        ctx.d_input_async[1], ctx.d_block_async[1], n, itemsPerBlock);
    cudaMemcpyAsync(ctx.h_block_async[1], ctx.d_block_async[1],
                    blocks * sizeof(float), cudaMemcpyDeviceToHost, ctx.streams[1]);

    // Arrays 2 and 3 follow the same pattern (min, max) ...

    // Wait for all streams before reading results
    for (int i = 0; i < 4; i++)
        cudaStreamSynchronize(ctx.streams[i]);
}
```

Listing 3: Asynchronous workflow (excerpt for sum and prod)

To ensure fair and reproducible benchmarks, all GPU and CPU memory is allocated once per array size outside the timed loop.

2 Results

Note: all results were verified by comparing the synchronous and asynchronous outputs, which matched the CPU result across all array sizes. To reduce noise, each array size was measured over 40 runs with one preceding warm-up run.

Table 1: Average execution time and speedup for the synchronous and asynchronous approaches at selected array sizes, averaged over 40 runs.

Array size N	Sync avg (ms)	Async avg (ms)	Speedup
64	≈ 0.12	≈ 0.06	2.04×
1 024	≈ 0.18	≈ 0.11	1.69×
4 096	≈ 0.41	≈ 0.28	1.47×
16 384	≈ 1.15	≈ 1.01	1.14×
65 536	≈ 2.10	≈ 1.90	1.11×
262 144	≈ 8.60	≈ 8.00	1.07×
1 048 576	≈ 47	≈ 43	1.10×

Figure 1 shows the average execution time of both approaches across all tested array sizes. The asynchronous approach is consistently faster, with the gap being most pronounced at small array sizes. Figure 2 plots the speedup ratio (synchronous time divided by asynchronous time) as a function of N. At small sizes ($N \leq 256$) the speedup reaches approximately 2×, it then decreases as N grows, plateauing near 1.1× for the largest arrays.

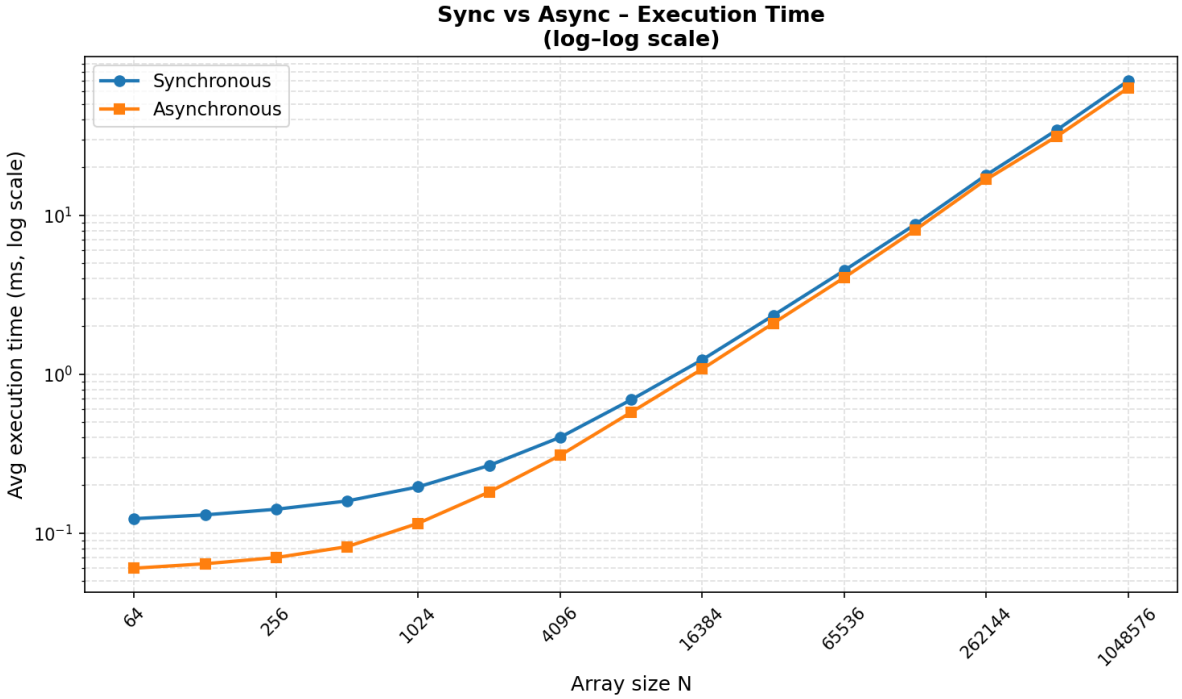


Figure 1: Average execution time (ms) vs. array size N for the synchronous and asynchronous approaches, averaged over 40 runs.

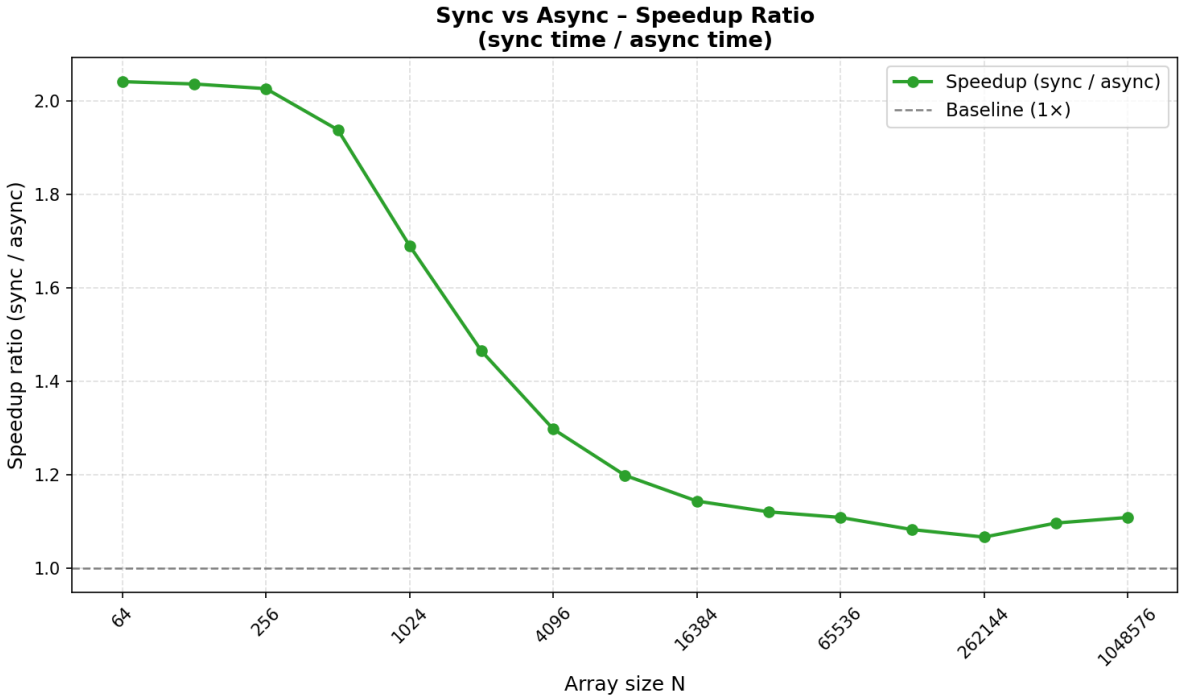


Figure 2: Speedup ratio (synchronous time / asynchronous time) vs. array size N.

3 Analysis

3.1 Why Asynchronous Execution is Faster

In the synchronous case, the CPU waits for a response from the GPU after every `cudaMemcpy` and kernel call. This means array generation and GPU processing are done strictly one after the other, with the CPU blocking during every GPU operation.

In the asynchronous case, the CPU offloads work to the GPU without blocking. While the GPU processes array 0, the CPU continues working on generating array 1. This way the CPU and GPU run in parallel with less blocking, hiding the array generation time behind GPU execution.

3.2 Observed Speedup and Limiting Factors

The speedup is highest ($\approx 2\times$) at small N and converges to $\approx 1\times$ at large N. To understand why, Table 2 and Figure 3 show the CPU array generation time as a percentage of the total execution time.

Table 2: CPU array generation time as a percentage of total execution time, for selected array sizes (averaged over 40 runs).

N	Sync total (ms)	Sync CPU gen (%)	Async total (ms)	Async CPU gen (%)
64	0.172	10.1	0.083	20.2
512	0.266	34.2	0.170	52.2
1 024	0.391	47.3	0.288	65.3
4 096	0.795	74.6	0.695	84.9
16 384	2.215	87.9	2.069	94.8
65 536	7.769	91.9	7.308	97.4
1 048 576	113.567	92.9	106.531	98.4

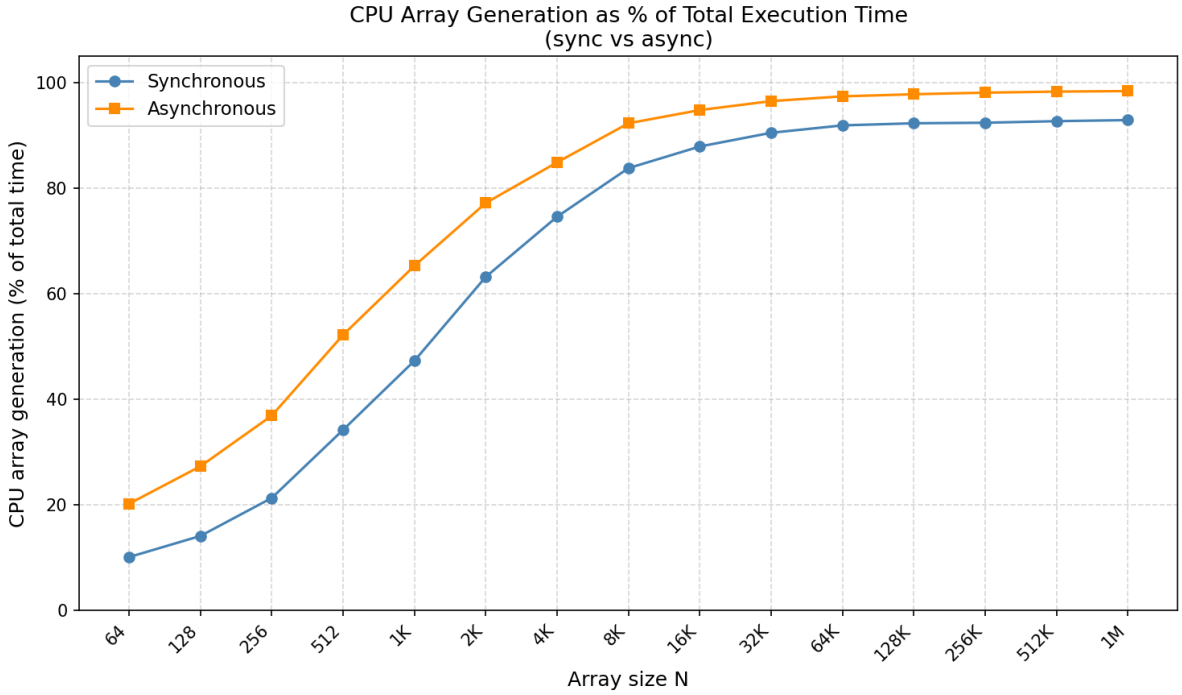


Figure 3: CPU array generation as a percentage of total execution time vs. array size N, for the synchronous and asynchronous approaches.

At small N, the CPU operations are short, and the synchronous operations spend most of their time blocking. So If the CPU and GPU can run in parallel, it will save a large fraction of the total time, leading to

the highest speedup.

As N grows, the CPU generation becomes the dominant cost, accounting for over 90% of the total time at large N as shown in Table 2 and Figure 3. At that point, the GPU is so fast that there is way less blocking to remove. Both approaches are limited by the CPU, so the speedup flattens near the $1\times$, and the two lines converge.

The CPU percentage appears higher in the asynchronous mode than in the synchronous mode because the total time is smaller, not because the CPU does more work, the raw generation times are nearly identical.

References

[1] Anthropic. (2024). *Claude*. Large language model. Throughout this project, the model was utilised as an assistant for C++/CUDA code generation, debugging, clarification of technical concepts, and the structuring and refinement of the written report.